Table 1-3. *`cli` SAPI directives – differences*

`implicit_flush`	`true`	`cli` programs often need to display output immediately
`max_execution_time`	0 (unlimited)	`cli` programs might need to run for minutes or even hours
`register_argc_argv`	`true`	Needing to access to cli arguments is usually a given
`output_buffering`	`false`	A compliment to `implicit_flush`, buffering output is usually the exception not the rule
`max_input_time`	`false`	Not necessary because there is no client data pending when the program is first invoked

Outputting Data

In this section we're going to be looking in detail at the mechanisms that PHP has for obtaining data from external sources – i.e., data that isn't literally declared within the program code itself. Chapter 2, "Data, Types, and Variables," will break down exactly how data representation works in PHP, how the ones and zeroes of binary bits – packaged into 8-bit bytes – are used to represent numbers, text, and so on. That chapter will also show you how to store and manipulate data as your program executes. All of these parts of the book contain code examples that make extensive use of data output mechanisms (functions and language constructs) that will show the results of the code (usually as `cli` SAPI programs), so before we go any further, we should look at how these output mechanisms work. Here are the most common ways used to generate program output in this book:

Table 1-4. *CLI output mechanisms*

`print, echo`	Send data to the output buffer as a text "string"
`print_r()`	Converts data to a human readable string, can send to output or return for use as variable
`var_dump()`	Similar to `print_r()`, but gives more comprehensive information about data types